

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

FairPlay

a fair distribution platform

Autore:
Gabriele Moschella

N° Matricola:
7133893

Corso principale:
Ingegneria del software

Docente corso:
Enrico Vicario

Indice

1 TODO CLEAN ACHITECTURE, IA DISCLOSURE **4**

2 Introduzione **4**

3 Architetture, strumenti e pratiche utilizzate **4**
3.1 Architettura del sistema 4
3.1.1 Livelli architetturali 4
3.1.2 Design e architectural pattern utilizzati 5
3.2 Strumenti e tecnologie 6
3.3 Pratiche di sicurezza 6

4 Analisi dei requisiti **6**
4.1 Definizone degli attori 6
4.2 Requisiti funzionali 7
4.3 Requisiti non funzionali 7

5 Progettazione del sistema **7**
5.1 Use case 7
5.2 Use case templates 7
5.3 Diagramma di navigazione 15
5.4 Mockup interfaccia utente 15

6 Modellazione dei Dati e della Persistenza **22**
6.1 Schema logico 22
6.2 Diagramma Entità-Relazione 22
6.3 Gestione della Connessione 24
6.4 Inizializzazione e Mappatura Relazionale 24

7 Architettura del Sistema **25**
7.1 Diagramma dei package 25
7.1.1 Espansione dei Packages 25
7.1.2 Domain 26
7.1.3 Application 27
7.1.4 Persistence 28
7.1.5 Presentation 29
7.1.6 Diagramma completo 29
7.2 Diagrammi di sequenza 30
7.2.1 Caso d'Uso: Approvazione Gioco 30
7.2.2 Caso d'Uso: Acquisto di un Gioco 30
7.2.3 Caso d'Uso: Rilascio di una Recensione 30

8 Tests **32**
8.1 Unit Testing 32
8.2 Integration Testing 33

9 Codice e snippets **34**
9.1 Application 34
9.2 Persistence 34
9.3 Presentation 35

Elenco delle figure

1 Diagramma use case 8
2 Mockup interfaccia Login 16
3 Mockup interfaccia Sign-Up 16
4 Mockup interfaccia Catalog 17
5 Mockup interfaccia Game 17
6 Mockup interfaccia Library 18
7 Mockup interfaccia Review 18
8 Mockup interfaccia Submit game 19
9 Mockup interfaccia Statistics 19
10 Mockup interfaccia Manage game 20
11 Mockup interfaccia Manage acceptance 20
12 Mockup interfaccia Manage removal 21
13 Mockup interfaccia Manage review 21
14 Diagramma Entità-Relazione (E-R) della piattaforma. 23
15 Diagramma packages del sistema. 25
16 Diagramma package domain. 26
17 Diagramma package application. 27
18 Diagramma package persistence. 28
19 Diagramma package presentation. 29
20 Diagramma completo. 30
21 Diagramma di sequenza dell'approvazione. 31
22 Diagramma di sequenza dell'acquisto. 31
23 Diagramma di sequenza della recensione. 32

1 TODO CLEAN ACHITECTURE, IA DISCLOSURE

2 Introduzione

Il monopolio nella distribuzione digitale dei videogiochi da parte di poche piattaforme di distribuzione (*Steam* ecc.) costringe spesso gli sviluppatori, in particolare studi indie, a cedere una percentuale 'onerosa' dei propri ricavi a queste piattaforme del settore. Questa situazione crea delle barriere all'ingresso per nuovi sviluppatori e si ripercuote indirettamente anche sui consumatori (i giocatori), limitando la visibilità di titoli di nicchia a favore di prodotti di alto budget (es. giochi tripla A). Alcune piattaforme, come l'*Epic Games Store*, hanno cercato di arginare il problema proponendo commissioni più eque e politiche maggiormente orientate agli sviluppatori. Anche se questo approccio ha parzialmente ridotto il monopolio, il mercato necessita ancora di alternative 'low cost' (non in termini di piattaforma, ma di politiche/commissioni), trasparenti e focalizzate sull'utente. L'obiettivo è proporre un'architettura software semplice ma efficace, per realizzare una piattaforma di distribuzione videoludica equa, intuitiva, frugale e costo-efficiente.

3 Architetture, strumenti e pratiche utilizzate

Per la progettazione e la realizzazione del sistema si è adottato uno stack tecnologico basato su standard, pattern architetturali e metodologie orientate alla manutenibilità e scalabilità del sistema.

3.1 Architettura del sistema

L'architettura del sistema è stata progettata seguendo i principi della **Layered Architecture**, assieme con il paradigma **Model-View-Controller (MVC)** per la gestione della separazione di responsabilità.

L'obiettivo primario di questa scelta architetturale è garantire indipendenza tra la logica, l'interfaccia e la persistenza, permettendo quindi di isolare i vari livelli del sistema e rendendo applicazione manutenibile, estensibile e testabile.

3.1.1 Livelli architetturali

Il sistema si sviluppa su quattro livelli indipendenti, regolati dalla *Dependency Rule* (le dipendenze puntano esclusivamente verso l'interno, ovvero verso il Dominio):

1. **Domain layer (dominio):** Rappresenta il fulcro dell'applicazione e contiene le entità fondamentali (`User`, `Player`, `Publisher`, `Admin`, `Game`, `Purchase`, `Review`) e relative enumerazioni (`RoleEnum`, `GameStateEnum`). In questo livello sono definite le regole pure, prive di dipendenze da framework, librerie o dettagli della persistenza.

2. **Application layer (logica):** Contiene la logica applicativa divisa in classi *Service* dedicate ad ogni servizio (`AuthenticationService`, `CatalogService`, `CheckoutService`, `ReviewService`). Questo livello si occupa di coordinare il flusso dei dati da e verso le entità del dominio e di richiamare le interfacce di persistenza (`UserDAO`, `GameDAO`, `PurchaseDAO`, `ReviewDAO`). Rispettando il principio di *Dependency Inversion* (DIP),

i Servizi non dipendono mai dalle implementazioni concrete del database (implementazione molto utile anche per facilitare l'uso di Mock durante la fase di *Unit Testing*).

3. **Persistence layer (persistenza):** Contiene l'implementazione delle interfacce richiamate a livello Application (`UserDAOImpl`, `GameDAOImpl`, `PurchaseDAOImpl`, `ReviewDAOImpl`) utilizzando la tecnologia **JDBC** e il driver `mysql-connector-j`. Questo livello gestisce la comunicazione con il DBMS MySQL, l'esecuzione delle query SQL e la mappatura del result set in oggetti del Dominio.

4. **Presentation layer (presentazione):** Contiene e gestisce l'interazione con l'utente. Cattura l'input da riga di comando (CLI), valida l'input e delega l'esecuzione dei casi d'uso ai relativi servizi del livello Application, restituendo feedback testuali all'utente.

3.1.2 Design e architectural pattern utilizzati

Per risolvere problematiche di progettazione object-oriented e garantire codice pulito e modulare, sono stati implementati i seguenti design pattern dellla **GoF** (Gang of Four):

- **Data Access Object (DAO) Pattern:** Utilizzato per astrarre e isolare completamente la logica di accesso al database dal resto dell'applicazione. Avendo separato le interfacce astratte (es. `GameDAO`) dalle loro implementazioni concrete (`GameDAOImpl`), il resto dell'applicazione interagisce con le entità di dominio senza conoscere tabelle, query o il database sottostante.

- **Singleton Pattern:** Applicato alla classe `DatabaseConnection` per la creazione della connessione al database. Il pattern garantisce l'esistenza di un'unica istanza della connessione per l'intero ciclo di vita dell'applicazione, in più per garantire sicurezza in ambienti multi-thread,il metodo `getInstance()` è stato reso *thread-safe*. In questo modo la connessione rimane unica e si evita di creare connessioni multiple.

- **Strategy Pattern:** Utilizzato nel processo di pagamento per la gestione dei diversi tipi di pagamento. La logica di transazione è astratta dall'interfaccia `PaymentStrategy`, consentendo di definire molteplici tipi di pagamento (es. Carte di Credito, PayPal). La classe `CheckoutService` delega la fase di pagamento alla strategia selezionata. Cosi facendo in futuro sarà possibile aggiungere nuove opzioni di pagamento senza modificare alcuna riga del servizio di Checkout.

- **State / Status Management Pattern:** Utilizzato per gestire il ciclo di vita dell'entita **Game**. La risorsa transita attraverso stati definiti dall'enum `GameStateEnum` (`PENDING` → `APPROVED`/`REJECTED`).

- **Model-View-Controller (MVC):** Utilizzato per separare le responsabilità di: *Model* che è rappresentato dalle entità di Dominio, *View* dalle componenti grafiche/console che mostrano le informazioni, e *Controller* dalle classi di coordinamento che catturano i comandi e invocano i metodi dei Servizi. In questo specifico caso il layer Presentation accorpa View e Controller, poiché la separazione in classi diverse per una semplice grafica a CLI avrebbe solo fatto aumentare le righe di codice inutilmente.

3.2 Strumenti e tecnologie

- **Linguaggio di programmazione**: **Java** (JDK 17+) sfruttando la programmazione orientata agli oggetti (OOP), incapsulamento, ereditarietà e polimorfismo.
- **DBMS**: **MySQL** per la persistenza relazionale dei dati, interfacciato tramite l'API nativa **JDBC** (*Java Database Connectivity*).
- **Modellazione e documentazione**: **Draw.io**, **PlantUML** e **Planttext** per i vari diagrammi strutturali e di comportamento; LaTeX per la stesura della documentazione tecnica e **Lovable** per i *mockup* dell'interfaccia. Utilizzata per la generazione di codice boilerplate, dei template per le classi (in particolare quelle di dominio) e della struttura di base dei test, successivamente modificati e adattati manualmente
- **Intelligenza artificiale**: Utilizzata per la generazione di codice *boilerplate*, dello scheletro per le classi, in particolare quelle di dominio (le quali sono formate solo dallo scheletro), dichiarazioni di attributi, costruttori, firme dei metodi e dell'impalcatura iniziale dei test, successivamente implementati e adattati manualmente, e creazione del file `init.sql` per l'inizializzazione del database (`CREATE TABLE...`) (**Gemini 3.1 Pro**); per velocizzare la fase di coding (**Copilot** all'interno dell'*IDE* **Intellij**). Utilizzata per consultazione per diagnosi e risoluzioni di errori emersi durante la fase di scrittura del codice e per risoluzione di errori durenate la fase di scrittura in LaTeX (errori di compilazione e formattazione).

3.3 Pratiche di sicurezza

- **Segregazione delle credenziali**: utilizzo di variabili d'ambiente di sistema per evitare l'hardcoding dei dati sensibili del database nel codice sorgente.
- **Integrità e normalizzazione**: uso della terza forma normale (3NF) e vincoli integrativi a livello di DB (`CHECK`, `FOREIGN KEY ON DELETE CASCADE`).

4 Analisi dei requisiti

4.1 Definizone degli attori

Segue la definizione dei ruoli e delle relative interazioni col sistema:

- Giocatore (**Player**): Utente finale che ha la possibilità di esplorare il negozio, acquistare videogiochi e aggiungerli alla propria libreria personale e recensirli.
- Sviluppatore (**Publisher**): Utente che crea contenuti (videogiochi). Può pubblicare nuovi giochi, scegliere il prezzo applicando sconti e visualizzare le statistiche di vendita e modificare i dati dei giochi esistenti.
- Amministratore (**Admin**): Utente con privilegi massimi. Si occupa della moderazione della piattaforma, approvando o rifiutando i giochi pubblicati dai Publisher e gestendo le recensioni.

4.2 Requisiti funzionali

Questi sono i requisiti funzionali fondamentali su cui si basa l'applicazione:

- Il sistema deve permettere la registrazione e l'autenticazione degli utenti in base al loro ruolo.
- Il sistema deve permettere ai Player di cercare giochi, acquistarli, visualizzarli e salvarli in una libreria.
- Il sistema deve permettere ai Publisher di inserire nuovi giochi nel catalogo.

4.3 Requisiti non funzionali

- Linguaggio: Il sistema sarà sviluppata in Java, con interfaccia grafica realizzata in **CLI**
- Architettura: Il codice seguirà il pattern architetturale **MVC** (Model-View-Controller) per separare la logica di business dall'interfaccia grafica.
- Persistenza: I dati saranno salvati su un database relazionale **MySQL**. L'accesso ai dati avverrà tramite il pattern **DAO** (Data Access Object).
- Sicurezza: Le password degli utenti non saranno salvate in chiaro nel database, ma tramite funzione di hashing (**MD5**).

5 Progettazione del sistema

Questa sezione fornisce un'analisi dettagliata delle azioni principali a di ciascun attore all'interno del sistema. Per garantire chiarezza tecnica, ogni caso d'uso è illustrato tramite un proprio template cioè una tabella descrittiva e, ove applicabile, un *mockup* che rappresenta l'interfaccia grafica che sarebbe presentata all'utente.

5.1 Use case

Qui sono stati definiti i casi d'uso che descrivono come gli attori (**User**) interagiscono col sistema

5.2 Use case templates

I seguenti template sottolineano gli aspetti primari degli use case identificati all'interno del progetto.

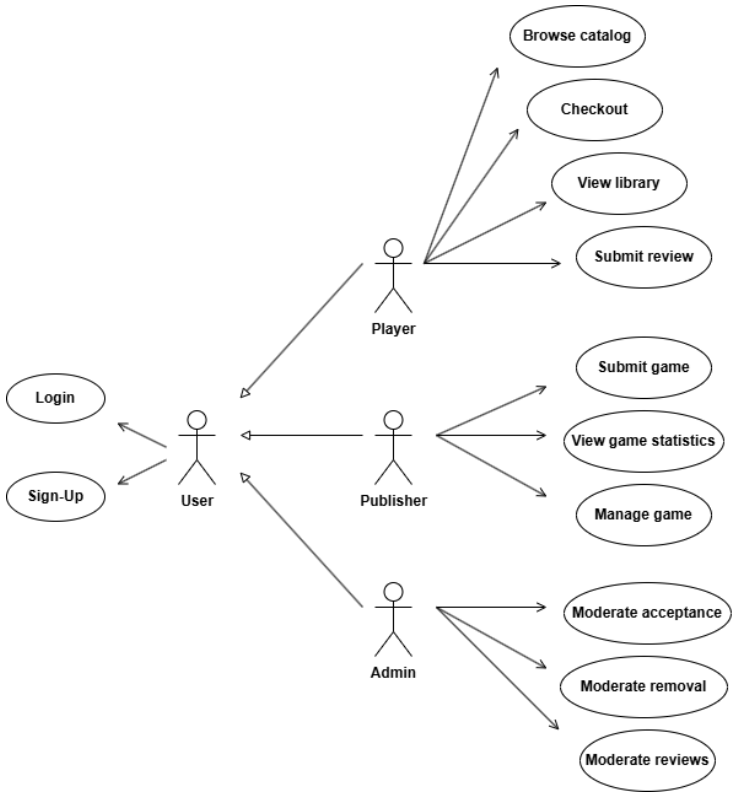


Figura 1: Diagramma use case

Use case n.1	Login
Brief description	L'utente accede al sistema attraverso le proprie credenziali
Level	User goal
Actors	User
Pre-conditions	L'utente ha un account registrato ma non ha ancora effettuato il login
Post-conditions	L'utente ha effettuato il login
Basic flow	1. L'utente inserisce la propria e-mail (Vedi mockup in figura 2) 2. L'utente inserisce la propria password 3. Il sistema convalida le credenziali di accesso 4. Il sistema permette all'utente di accedere
Alternative flow	• Se si verifica l'inserimento di credenziali incorrette, il sistema mostra un messaggio di errore invitando a riprovare

Tabella 1: Template dello use case Login

Use case n.2	Sign-Up
Brief description	L'utente si registra al sistema attraverso delle credenziali
Level	User goal
Actors	User
Pre-conditions	L'utente non ha un account registrato e non ha ancora effettuato il login
Post-conditions	L'utente ha effettuato la registrazione ed è ora disponibile per lui il login
Basic flow	1. L'utente inserisce la propria e-mail (Vedi mockup in figura 3) 2. L'utente inserisce la propria password 3. Il sistema convalida le credenziali di accesso 4. Il sistema permette all'utente di registrarsi
Alternative flow	• Se si verifica l'inserimento di dati incorretti, il sistema mostra un messaggio di errore invitando a riprovare

Tabella 2: Template dello use case Sing-Up

Use case n.3	Browse catalog
Brief description	Il player naviga attraverso il catalogo di videogiochi disponibili
Level	User goal
Actors	Player
Pre-conditions	Il Player deve aver effettuato il login
Post-conditions	Il Player visione e cerca all'interno del catalogo videogiochi
Basic flow	1. Il Player naviga il catalogo (Vedi mockup in figura 4) 2. Il Player visiona la pagina di un gioco (Vedi mockup in figura 5)
Alternative flow	• Se il gioco cercato è inesistente, il sistema mostra un messaggio di errore invitando a provare con un altro nome

Tabella 3: Template dello use case Browse catalog

Use case n.4	Check out
Brief description	Il player effettua un acquisto previa pagamento
Level	User goal
Actors	Player
Pre-conditions	Il Player deve aver effettuato il login e scelto il gioco da comprare
Post-conditions	Il Player ha effettuato un acquisto e il gioco sarà visualizzabile all'interno della libreria
Basic flow	1. Il Player sceglie il videogioco 2. Il Player preme il pulsante di acquisto (Vedi mockup in figura 5) 3. Il Player sceglie il metodo di pagamento 4. Il Player inserisce i dati di pagamento 5. Il Player conferma l'acquisto
Alternative flow	• Se il gioco risulta già acquistato, il sistema notificherà la presenza del gioco in libreria • Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 4: Template dello use case Checkout

Use case n.5	View library
Brief description	Il player visiona la libreria personale
Level	User goal
Actors	Player
Pre-conditions	Il Player deve aver effettuato il login ed essere nella sezione libreria
Post-conditions	Il Player avrà visionato i giochi precedentemente acquistati
Basic flow	1. Il Player entra nella sezione libreria 2. Il Player visiona i propri giochi (Vedi mockup in figura 6)
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 5: Template dello use case View library

Use case n.6	Submit review
Brief description	Il player crea una recensione
Level	User goal
Actors	Player
Pre-conditions	Il Player deve aver effettuato il login, essere nella pagina del videogioco e averlo precedentemene acquistato
Post-conditions	Il Player avrà pubblicato la sua recensione
Basic flow	1. Il Player entra nella pagina del videogioco 2. Il Player scrive una recensione (Vedi mockup in figura 7) 3. Il Player pubblica la recensione
Alternative flow	• Sei il Player non possiede il videgioco, il sistema mostra un messaggio di errore che lo informa del mancato acquisto dello stesso. • Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 6: Template dello use case Submit review

Use case n.7	Submit game
Brief description	Il publisher pubblica un gioco
Level	User goal
Actors	Publisher
Pre-conditions	Il Publisher deve aver effettuato il login
Post-conditions	Il Publisher avrà pubblicato il videogioco o potrà visionaro all'interno della sua lista di giochi pubblicati
Basic flow	1. Il Publisher inserisce i dati del videogioco (Vedi mockup in figura 8) 2. Il Publisher conferma i dati
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 7: Template dello use case Submit game

Use case n.8	View game statistics
Brief description	Il publisher visiona le statistiche di un gioco da lui pubblicato
Level	User goal
Actors	Publisher
Pre-conditions	Il Publisher deve aver effettuato il login ed essere nella pagina del gioco
Post-conditions	Il Publisher avrà visionato le statistiche del gioco scelto
Basic flow	1. Il Publisher sceglie il videogioco 2. Il Publisher visiona le statistiche (Vedi mockup in figura 9)
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 8: Template dello use case View Sales Statistics

Use case n.9	Manage game
Brief description	Il publisher gestisce le informazioni dei propri videogiochi
Level	User goal
Actors	Publisher
Pre-conditions	Il Publisher deve aver effettuato il login ed essere nella pagina del gioco
Post-conditions	Le informazioni del gioco risulteranno modificate all'interno del catalogo
Basic flow	1. Il Publisher sceglie il videogioco 2. Il Publisher modifica i dati (Vedi mockup in figura 10)
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 9: Template dello use case Manage game

Use case n.10	Moderate acceptance
Brief description	L'admin gestisce l'accettazione dei videogiochi
Level	User goal
Actors	Admin
Pre-conditions	L'admin deve aver effettuato il login ed essere nella sezione di del videogioco
Post-conditions	Il gioco sarà accettato (pubblicato sul catalogo)
Basic flow	1. L'Admin accede alla sezione dei videogiochi in attesa di accettazione (Vedi mockup in figura 11) 2. L'admin accede alla sezione del videogioco visionandonele informazioni 3. L'Admin accetta la pubblicazione del videogioco
Alternative flow	• L'Admin rifiuta la pubblicazione del videogioco • Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 10: Template dello use case Moderate acceptance

Use case n.11	Moderate removal
Brief description	L'admin gestisce la rimozione dei videogiochi
Level	User goal
Actors	Admin
Pre-conditions	L'admin deve aver effettuato il login ed essere nella sezione di rimozione dei videogiochi
Post-conditions	Il gioco sarà eliminato (rimosso dal catalogo)
Basic flow	1. L'Admin accede alla sezione dei videogiochi (Vedi mockup in figura 12) 2. L'admin rimuove il videogioco
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 11: Template dello use case Moderate removal

Use case n.12	Moderate review
Brief description	L'admin modera le recensioni da parte dei Player sui videogiochi eliminandole
Level	User goal
Actors	Admin
Pre-conditions	L'admin deve aver effettuato il login ed essere nella pagina del videogioco
Post-conditions	La recensiona è stata eliminata
Basic flow	1. L'Admin accede alla sezione del videogioco 2. L'Admin sceglie la recensione (Vedi mockup in figura 13) 3. L'Admin elimina la recensione
Alternative flow	• Se si verifica un errore di sistema, il sistema mostra un messaggio di errore invitando a riprovare più tardi

Tabella 12: Template dello use case Moderate review

5.3 Diagramma di navigazione

Trattandosi di un'interfaccia a riga di comando (**CLI**), la navigazione del sistema è strutturata come una macchina a stati finiti, quindi semplici menù a scelta multipla che permettono di navigare attraverso l'applicazione. L'accesso alle diverse sezioni dell'applicazione è vincolato all'autenticazione dell'utente e al ruolo associato associato a quest'ultimo.

- **Stato Iniziale (Unauthenticated)**:
  - `[1]` Login → Validazione credenziali → Reindirizzamento in base al *RoleEnum*.
  - `[2]` Sign-Up → Registrazione nuovo **Player**/**Publisher** → Ritorno al menù iniziale.
  - `[3]` Esci → Chiusura dell'applicazione.
- **Dashboard Player** (se `role == PLAYER`):
  - `[1]` Sfoglia Catalogo → Selezione Gioco → *CheckoutFlow* (Scelta *PaymentStrategy*) → Salvataggio Acquisto.
  - `[2]` Lascia una recensione → Selezione Gioco Posseduto → Inserimento Recensione (`leaveReview()`).
  - `3` I miei acquisti → Visiona gli acquisti
  - `[4]` Logout → Ritorno allo stato iniziale.
- **Dashboard Publisher** (se `role == PUBLISHER`):
  - `[1]` Pubblica gioco → Inserimento dati → Creazione gioco in stato *PENDING*.
  - `[2]` Visualizza giochi → `browseGames()` → Scegli gioco da visionare → Recensioni / Modifica / Statistiche
  - `[3]` Logout → Ritorno allo stato iniziale.
- **Dashboard Admin** (se `role == ADMIN`):
  - `[1]` Valuta giochi in attesa → Selezione gioco → Approva (*APPROVED*) / Rifiuta (*REJECTED*).
  - `[2]` Moderazione recensioni → Selezione Recensione → Eliminazione.
  - `[3]` Moderazione videogiochi → Selezione gioco → Eliminazione.
  - `[4]` Logout → Ritorno allo stato iniziale.

5.4 Mockup interfaccia utente

Di seguito i *mockup* illustranti le principali interfacce grafiche dell'applicazione:

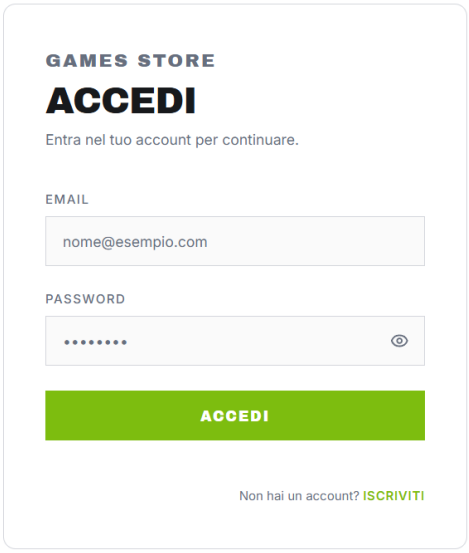


Figura 2: Mockup interfaccia Login

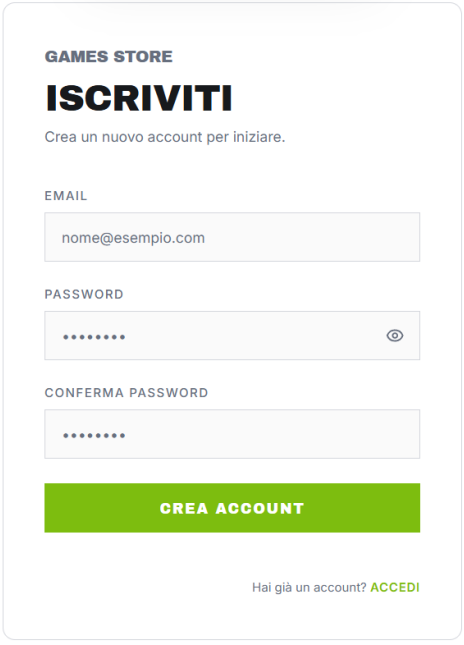


Figura 3: Mockup interfaccia Sign-Up

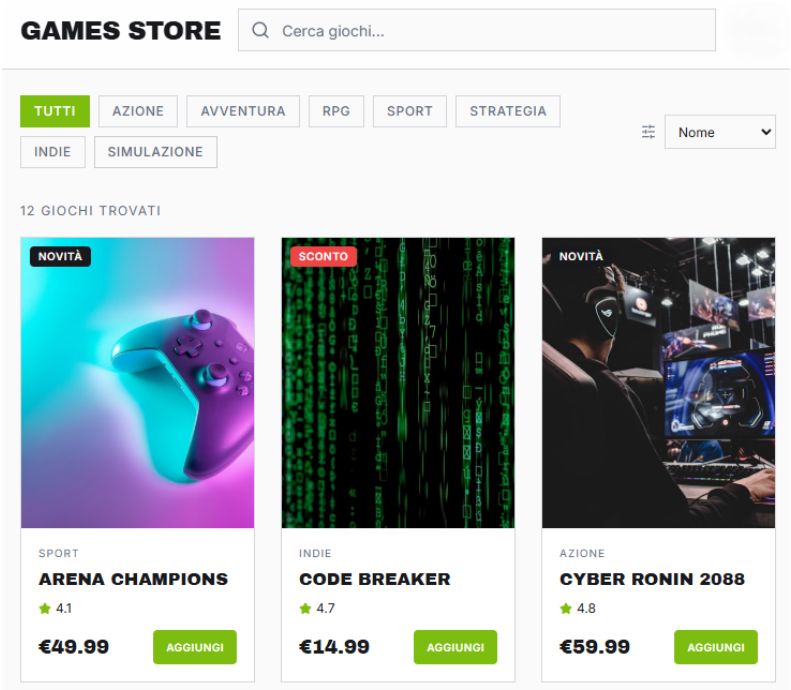


Figura 4: Mockup interfaccia Catalog

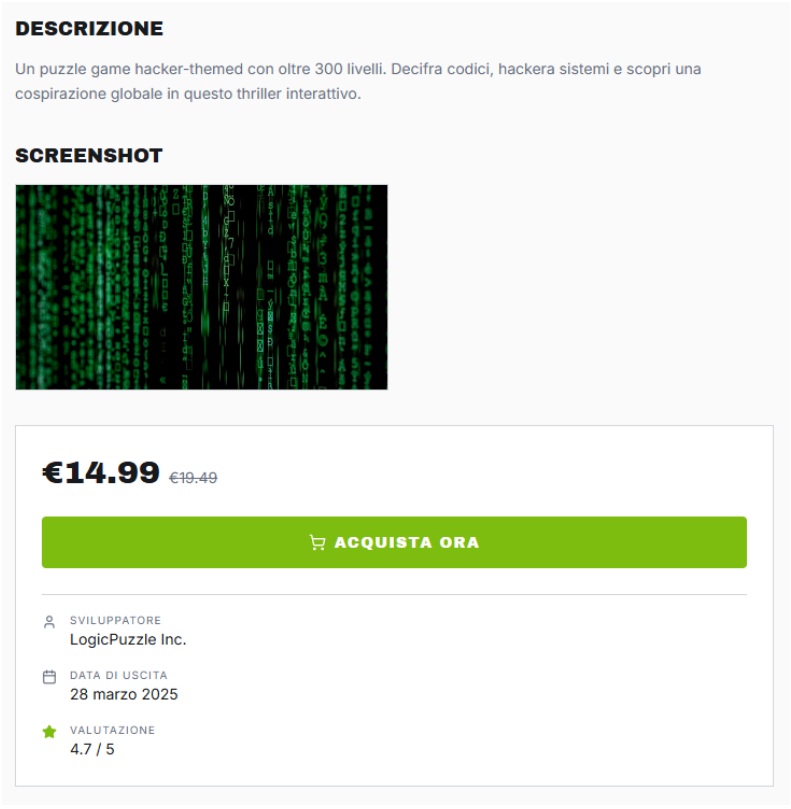


Figura 5: Mockup interfaccia Game

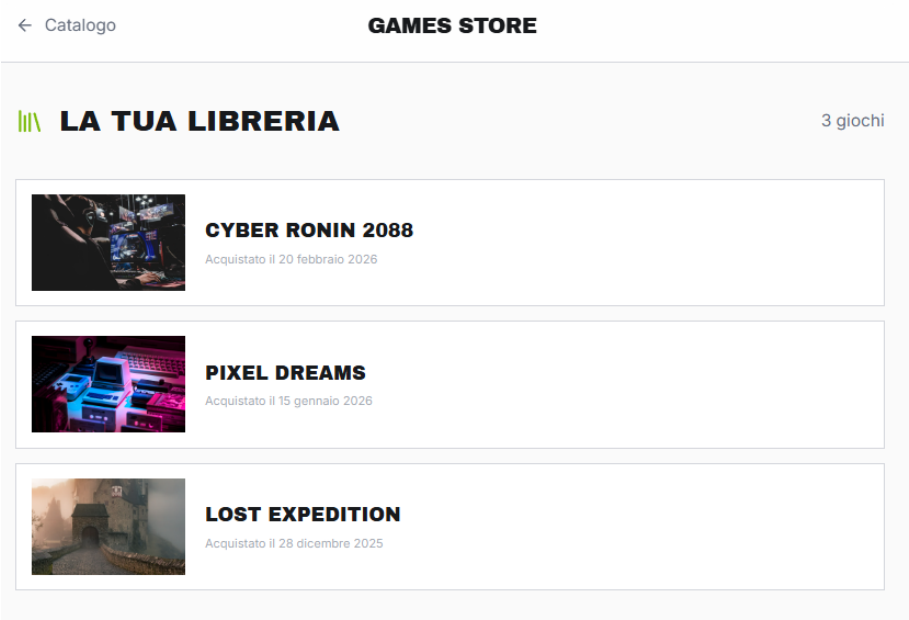


Figura 6: Mockup interfaccia Library

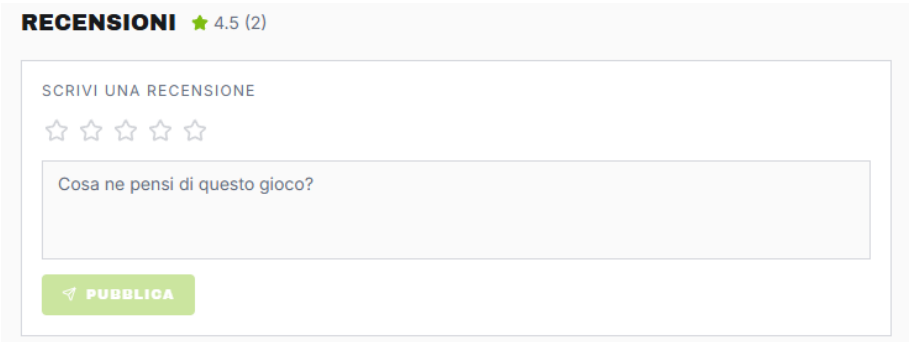


Figura 7: Mockup interfaccia Review



Figura 8: Mockup interfaccia Submit game

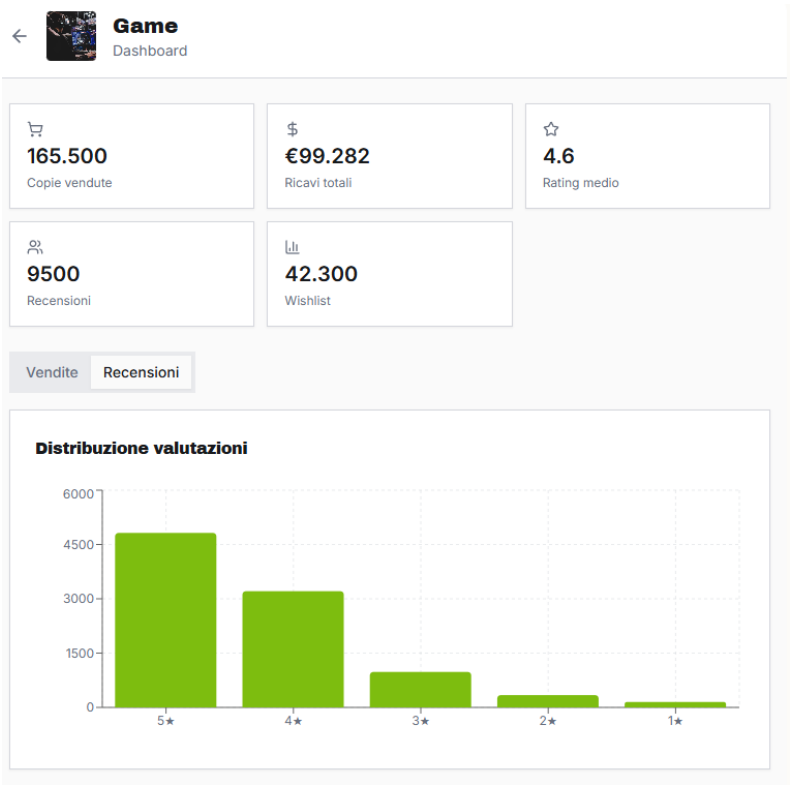


Figura 9: Mockup interfaccia Statistics

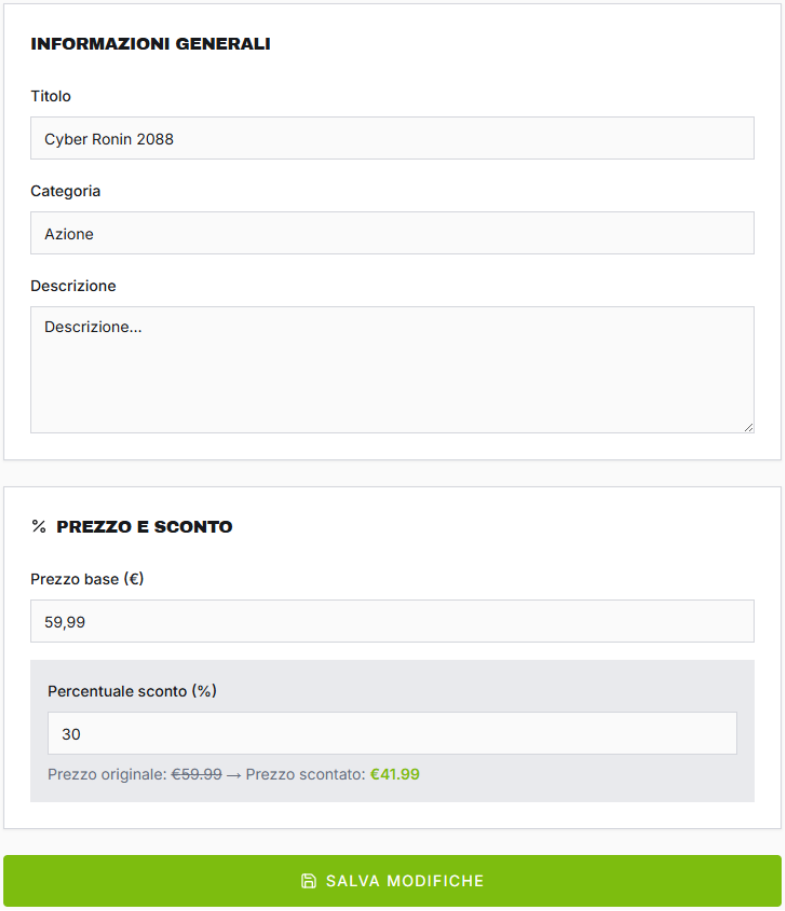


Figura 10: Mockup interfaccia Manage game

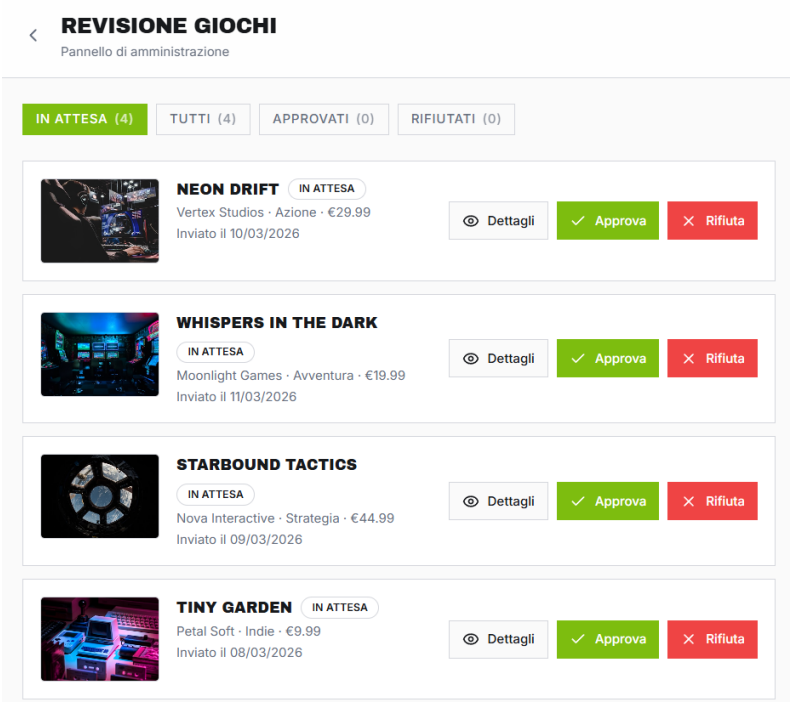


Figura 11: Mockup interfaccia Manage acceptance

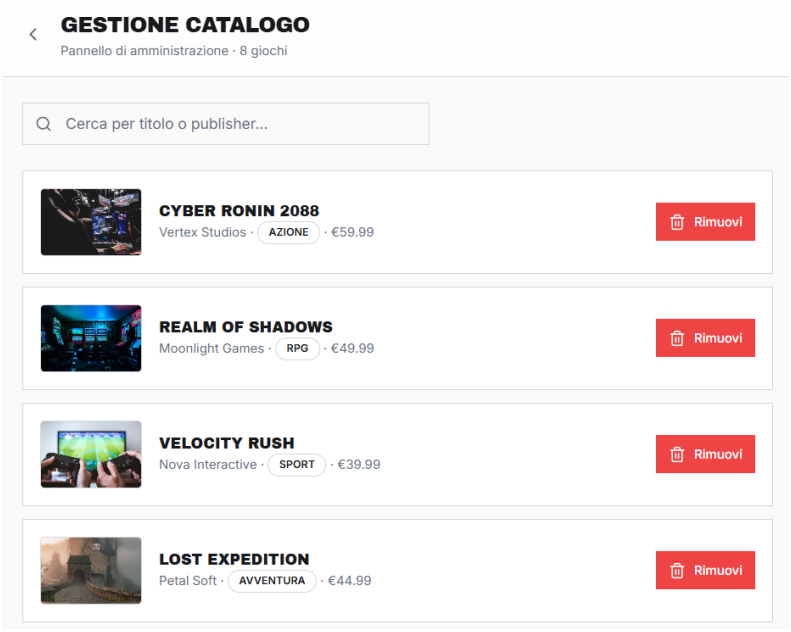


Figura 12: Mockup interfaccia Manage removal

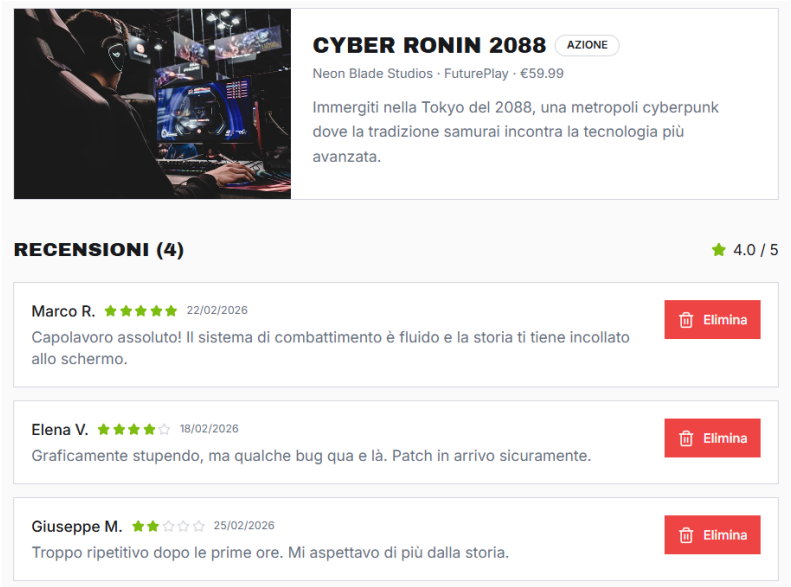


Figura 13: Mockup interfaccia Manage review

6 Modellazione dei Dati e della Persistenza

La modellazione della persistenza delle informazioni all'interno dell'applicazione è stata progettata per garantire integrità dei dati. Il modello concettuale dei dati è rappresentato visivamente tramite il Diagramma Entità-Relazione (*E-R*) in Figura 14 e testualmente tramite lo schema logico sottostante. L'architettura del database ruota attorno a entità centrali, strutturate sfruttando il concetto di generalizzazione per ottimizzare la gestione degli attori principali del sistema:

- **User**: Entità padre (superclasse), da questa derivano tre sottoclassi specializzate:
  - **Player**: L'utente finale, collegato tramite relazione *has* ai giochi che compra (presenti nella propria libreria)
  - **Publisher**: L'utente creatore, collegato tramite la relazione *publishes* ai giochi che inserisce a sistema.
  - **Admin**: L'utente moderatore, con privilegi di supervisione.
- **Game**: Rappresenta il prodotto digitale. Contiene i metadati essenziali (*title, description, price, discount*) e uno *state* (stato di approvazione) gestito dagli amministratori.
- **Library**: Entità associata a un singolo *Player* (relazione 1 a 1) che memorizza i giochi acquistati da quest'ultimo e si relaziona con l'entità *Game* attraverso una relazione molti-a-molti *contains*.
- **Review**: Memorizza le recensioni degli utenti. È caratterizzata da un *vote* (voto numerico), un *text* e una *date*. Relaziona in modo univoco il *Player* che l'ha scritta e il *Game* recensito.

6.1 Schema logico

- **User**(<u>id</u>, email, password, username, role)
- **Game**(<u>id</u>, title, description, state, price, discount, *Publisher*)
- **Library**(<u>id</u>, date, *Player*, *Game*)
- **Review**(<u>id</u>, vote, text, date, *Player*, *Game*)

6.2 Diagramma Entità-Relazione

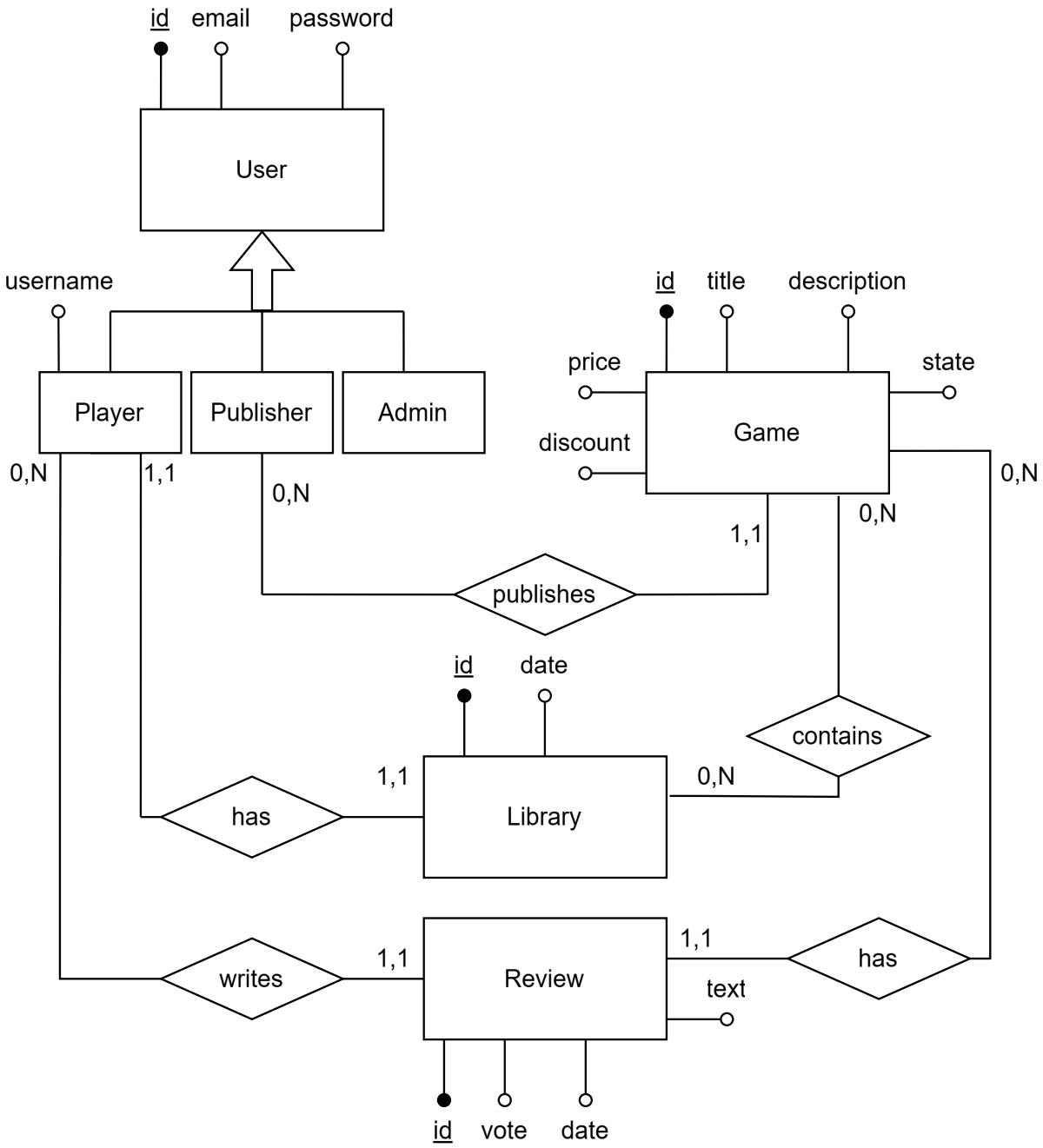


Figura 14: Diagramma Entità-Relazione (E-R) della piattaforma.

6.3 Gestione della Connessione

La connessione al database relazionale è gestita dalla classe `DatabaseConnection` all'interno del package `persistence`. La classe è strutturata applicando il pattern di progettazione **Singleton**, garantendo che un'unica istanza mantenga il controllo della connettività per l'intero ciclo di vita dell'applicazione. L'accesso a tale istanza avviene tramite il metodo sincronizzato `getInstance()`, assicurando la thread-safety in contesti di esecuzione concorrente.

La comunicazione con il database è regolata dal metodo `getConnection()`, che controlla preventivamente lo stato della risorsa: la connessione viene rigenerata unicamente se risulta nulla o chiusa. Per garantire flessibilità e sicurezza, le credenziali non sono hard-coded nel codice sorgente ma vengono recuperate dinamicamente dall'ambiente (variabili d'ambiente):

- `DB_URL`: Stringa di connessione JDBC nel formato `jdbc:mysql://host:port/database_name`.
- `DB_USER`: Utente del DBMS abilitato alla gestione dello schema.
- `DB_PASSWORD`: Credenziale di autenticazione.

Qualora una di queste variabili risulti mancante, il sistema blocca l'inizializzazione sollevando un'eccezione di tipo `IllegalStateException`.

6.4 Inizializzazione e Mappatura Relazionale

All'avvio del livello di persistenza, lo schema del database viene inizializzato automaticamente tramite l'esecuzione dello script `init.sql` collocato nella cartella delle risorse del progetto. L'operazione è strutturata per essere idempotente: ogni istruzione SQL sfrutta la clausola `CREATE TABLE IF NOT EXISTS`, garantendo che l'esecuzione dello script non sovrascriva i dati già presenti o le strutture già configurate.

Il passaggio dal modello Entità-Relazione alla struttura relazionale ha adottato le seguenti strategie:

- **Gestione dell'Ereditarietà (Single Table Inheritance)**: La gerarchia dell'entità astratta `User` e delle sue specializzazioni (`Player`, `Publisher`, `Admin`) è stata risolta mappando le entità su una singola tabella `users`. L'attributo discriminante `role` (`ENUM`) definisce la tipologia dell'utente.
- **Immutabilità del Prezzo negli Acquisti**: Per garantire la correttezza dello storico di un acquisto di fronte a variazioni del prezzo di listino o di sconti del videogioco, la relazione d'acquisto memorizza in modo permanente il campo `price_paid` calcolato al momento del checkout.
- **Vincoli di Integrità e Dominio**: La prevenzione di acquisti duplicati per lo stesso videogioco è garantita da un indice di univocità composto sui campi `(id_player, id_game)`. La validità dei dati è inoltre assicurata a livello DB tramite vincoli `CHECK` sulle percentuali di sconto ($0 \leq discount \leq 100$) e sulla valutazione delle recensioni ($1 \leq score \leq 5$).

7 Architettura del Sistema

7.1 Diagramma dei package

Il sistema è stato progettato adottando un'architettura a livelli ispirata ai principi della Layered Architecture. In questo modo si garantisce una rigorosa separazione delle responsabilità, disaccoppiando la logica di business dai dettagli implementativi come l'interfaccia utente o l'infrastruttura del database.

Il progetto software è stato strutturato nei seguenti macro-package:

- `presentation`: Costituisce il livello di interazione con l'utente. Per focalizzare lo sviluppo sull'architettura, si è optato per un'interfaccia a riga di comando (CLI). Questo package contiene le viste testuali e i *Controller* che interpretano gli input dell'utente, delegando l'esecuzione delle operazioni al livello applicativo.
- `application` **(Business Logic)**: È il livello che orchestra il flusso dei dati: riceve le richieste, coordina le entità di dominio e interagisce con il livello di persistenza. Praticamente il tramite tra interfaccia, elaborazioni e persistenza.
- `domain` **(Domain Logic)**: Rappresenta il nucleo puro del software. Al suo interno sono definite esclusivamente le Entità del dominio (come *Player*, *Game*, *Review*). Contiene la logica degli oggetti e le regole di validazione base, garantendo totale isolamento dal resto.
- `persistence` **(Data Layer)**: Gestisce la comunicazione con il database relazionale. Per l'accesso ai dati è stato adottato il pattern architetturale **DAO (Data Access Object)**, attraverso l'uso delle operazioni CRUD (Create, Read, Update, Delete). L'implementazione concreta dei DAO è realizzata tramite le API standard **JDBC (Java Database Connectivity)**, permettendo un controllo diretto sulle query SQL native, senza ricorrere a framework come ORM.

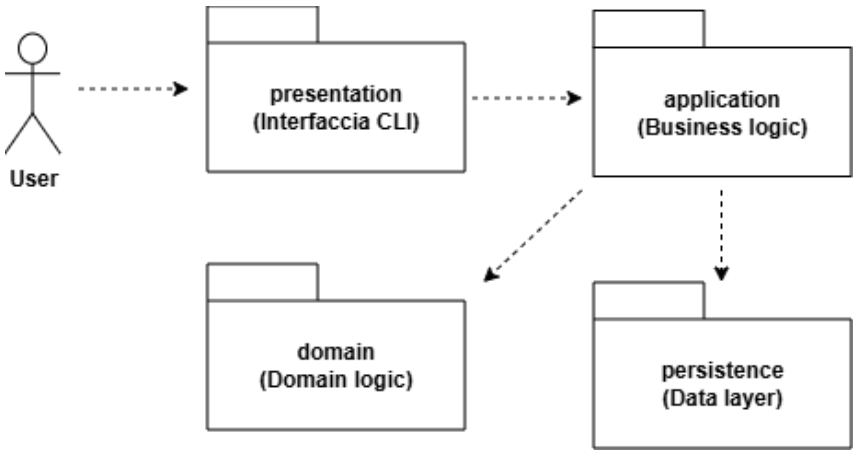


Figura 15: Diagramma packages del sistema.

7.1.1 Espansione dei Packages

Ogni package all'interno è formato da diverse classi che andiamo ad analizzare più nel dettaglio.

7.1.2 Domain

Il package nucleo del sistema contenente le classi Entità:

- `User` (Classe astratta) è l'entità che racchiude le informazioni generali di ogni utente e che viene poi ereditata da ogni 'attore'
  - `Player` (estende User) è l'entità che identifica il giocatore
  - `Publisher` (estende User) è l'entità che identifica il pubblicatore di giochi
  - `Admin` (estende User) è l'entità che identifica l'amministratore del sistema con privilegi
- `Game` rappresenta il prodotto vero e proprio
- `Purchase` rappresenta la 'ricevuta' e denota il possesso del gioco
- `Reviews` rappresenta la recensione
- `PasswordHasher` utility class (più che di dominio) che permette di encriptare la password

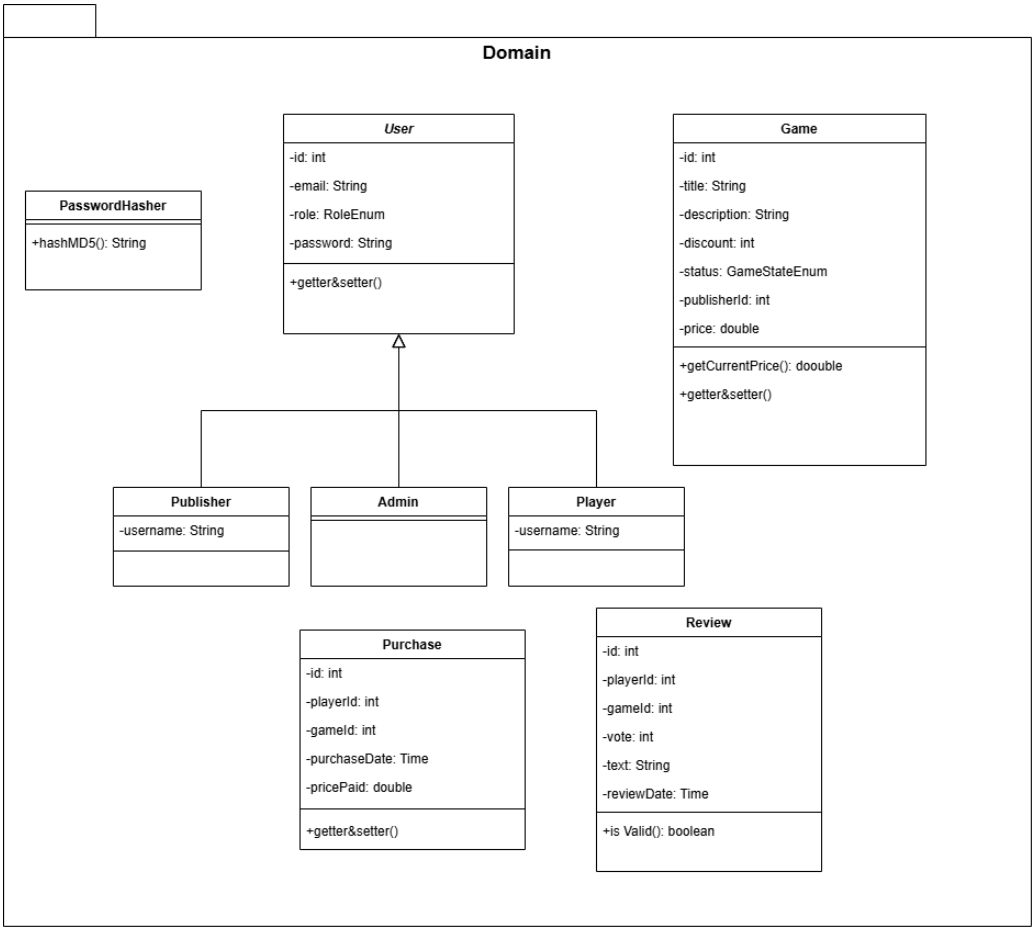


Figura 16: Diagramma package domain.

7.1.3 Application

Questo è il package core delle elaborazioni e dei servizi.

- `AuthenticationService` gestisce l'accesso al sistema.
- `CatalogService` gestisce tutto ciò che rigurda il catalogo dei videogiochi
- `CheckoutService` gestisce la terna utente, gioco, pagamento.
- `PaymentStrategy` (interfaccia) gestisce il metodo di pagamento desiderato.
  - `PayPalPayment` (implementa PaymentStrategy) gestisce il metodo di pagamento Paypal.
  - `CreditCardPayment` (implementa PaymentStrategy) gestisce il metodo di pagamento con carta.
- `ReviewService` gesistisce il sistema recensionistico

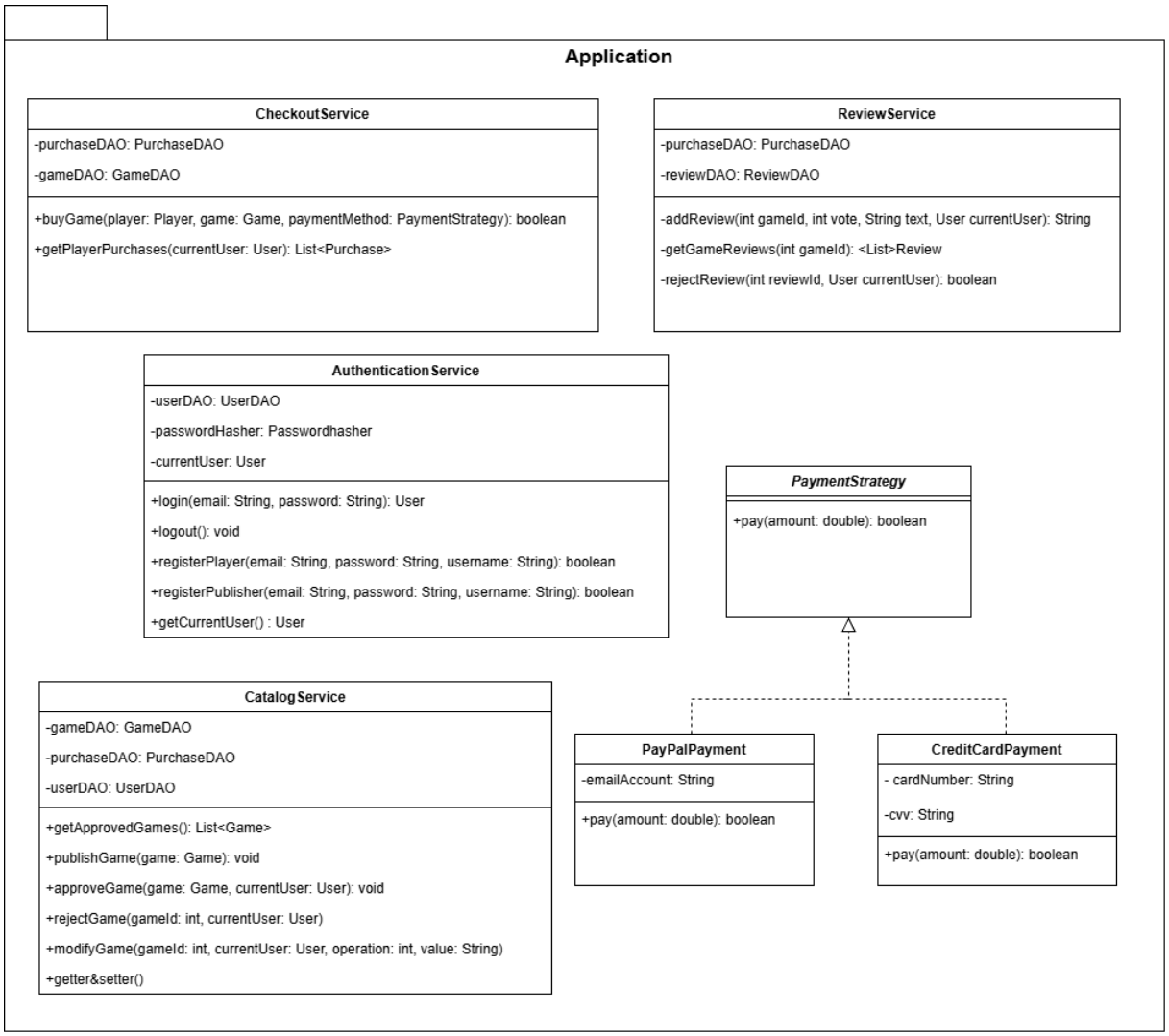


Figura 17: Diagramma package application.

7.1.4 Persistence

Questo è il package relativo al database, persistenza e struttura dei dati.

- `DatabaseConnection` gestisce la connessione al database.
- `UserDAO` (interfaccia) definisce le operazioni relative allo user
  - `UserDAOImpl` (implementa UserDao) definisce le query relative allo user
- `GameDAO` (interfaccia) definisce le operazioni relative al gioco
  - `GameDAOImpl` (implementa GameDao) definisce le query relative al gioco
- `PurchaseDAO` (interfaccia) definisce le operazioni relative all'acquisto
  - `PurchaseDAOImpl` (implementa PurchaseDao) definisce le query relative all'acquisto
- `ReviewDAO` (interfaccia) definisce le operazioni relative alle recensioni
  - `ReviewDAOImpl` (implementa ReviewDao) definisce le query relative alle recensioni

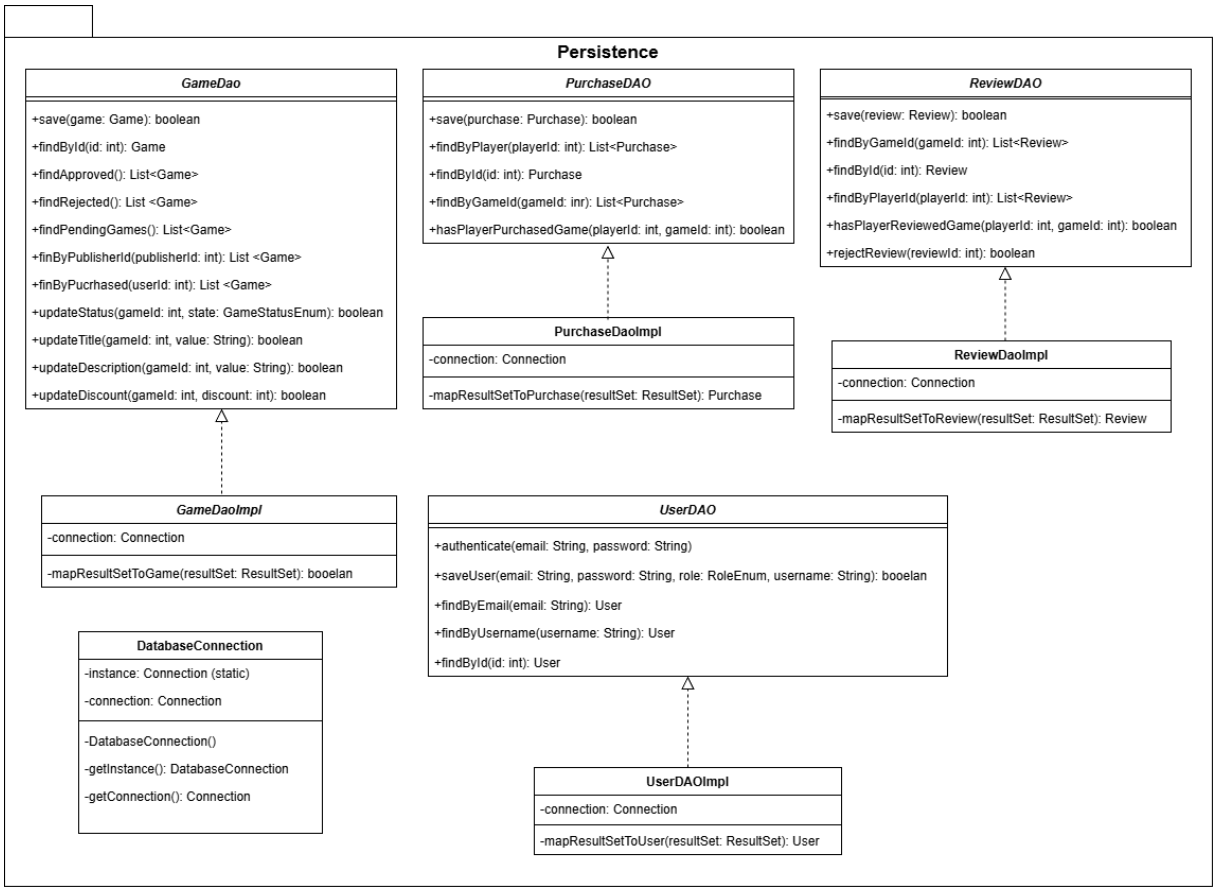


Figura 18: Diagramma package persistence.

7.1.5 Presentation

Questo è il package relativo alla presentazione, che si interfaccia con l'utente facendo da tramite per i Service all'interno della business logic.

- `AuthController` gestisce il menu iniizle (login/sign-up)
- `PlayerController` gestisce l'interfaccia del player
- `PublisherController` gestisce l'interfaccia del publisher
- `AdminController` gestisce l'interfaccia dell'admin

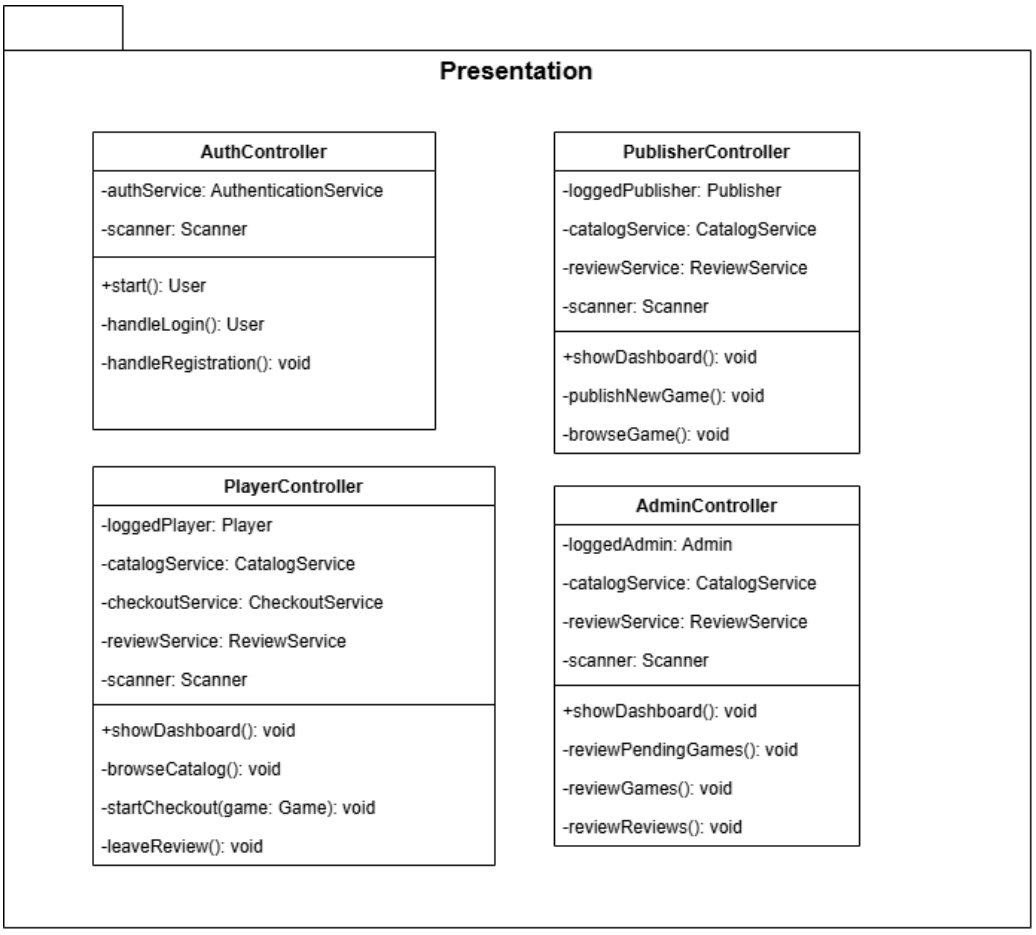


Figura 19: Diagramma package presentation.

7.1.6 Diagramma completo

Di seguito il diagramma completo dei package con le relative classe e relazioni.

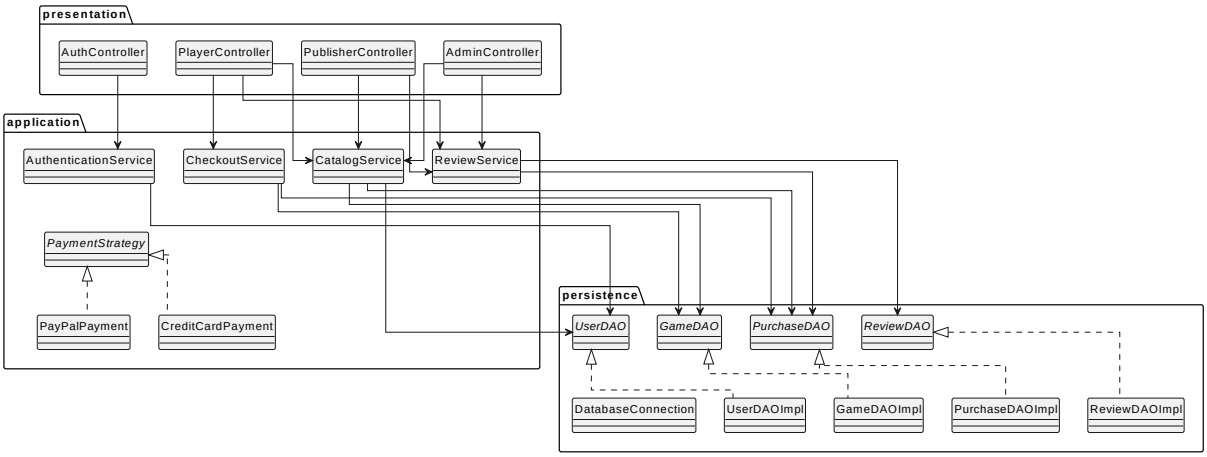


Figura 20: Diagramma completo.

7.2 Diagrammi di sequenza

Per illustrare il comportamento del sistema durante la normale esecuzione e lo scambio di messaggi tra i diversi livelli architetturali, vengono analizzati tre casi d'uso principali: acquisto di un gioco, approvazione di un gioco e rilascio di una recesione.

7.2.1 Caso d'Uso: Approvazione Gioco

L'amministratore seleziona la funzionalità di revisione. Il controller `AdminController` richiede al `CatalogService` la lista dei titoli con stato *PENDING*. Una volta selezionato il gioco, il servizio invoca il metodo di dominio `approve()` sull'entità `Game`, aggiornandone lo stato. Successivamente, la modifica viene resa persistente sul database tramite il metodo `updateStatus()` della classe `GameDAOImpl`.

7.2.2 Caso d'Uso: Acquisto di un Gioco

Il giocatore seleziona la procedura di checkout dal `PlayerController`. L'interfaccia istanzia la concreta strategia di pagamento (es. `CreditCardPayment`) scelta dall'utente e la passa al `CheckoutService`. Il servizio effettua il pagamento tramite l'interfaccia `PaymentStrategy`. Se la transazione va a buon fine, viene creata l'istanza dell'entità `Purchase` con il prezzo storico bloccato al momento dell'acquisto e salvata nel database mediante `PurchaseDAOImpl`.

7.2.3 Caso d'Uso: Rilascio di una Recensione

Il giocatore attraverso il metodo `leaveReview()`, inserisce punteggio (stelle) e testo della recensione. Il controller istanzia un oggetto `Review` e ne verifica la validità tramite il metodo `isValid()`. Superata la validazione, l'oggetto viene inviato al `ReviewDAOImpl` che esegue l'operazione di inserimento nel database.

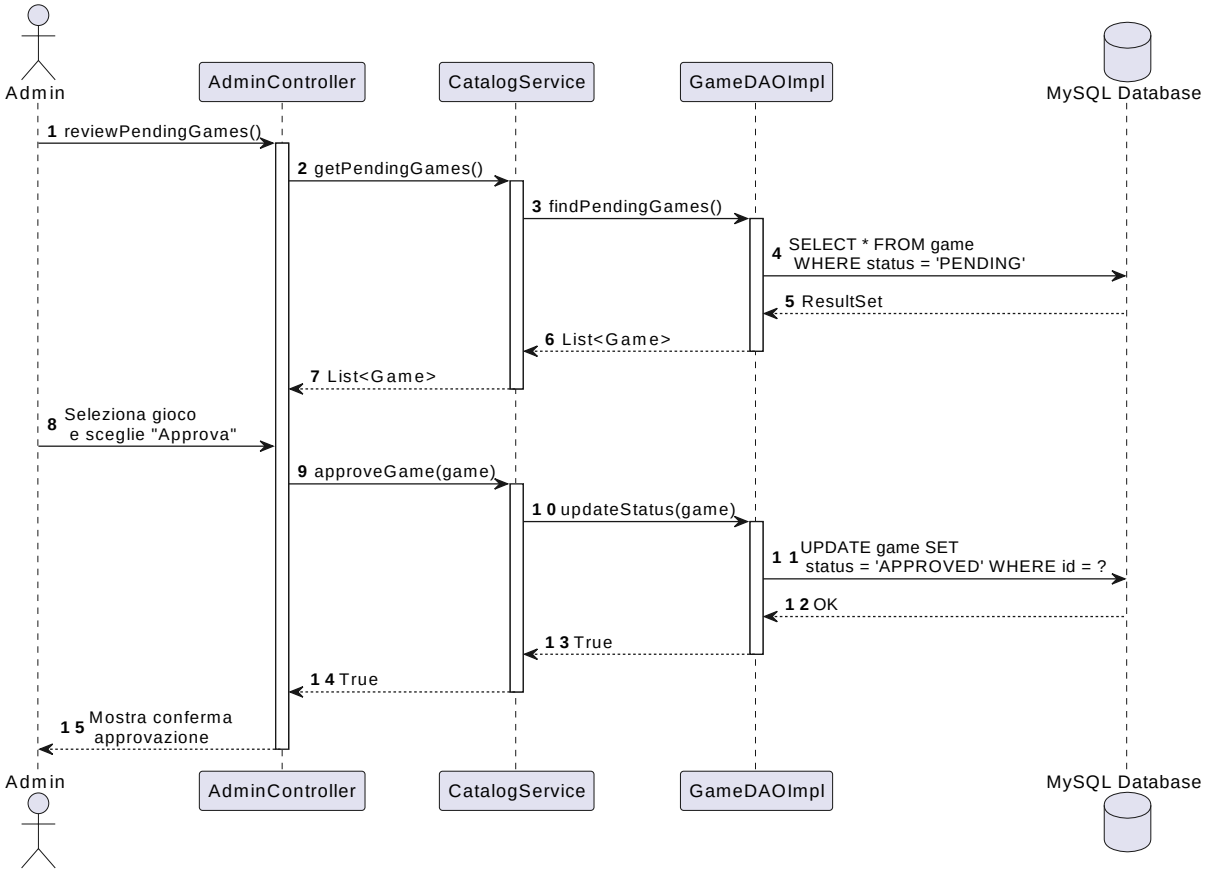


Figura 21: Diagramma di sequenza dell'approvazione.

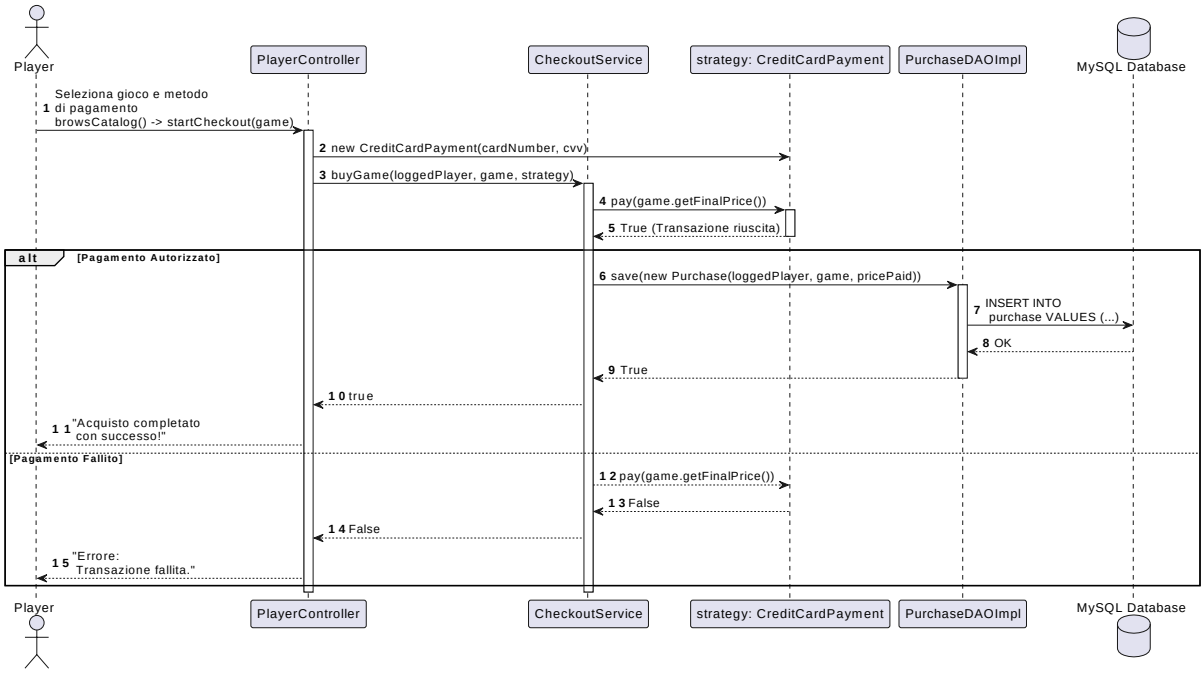


Figura 22: Diagramma di sequenza dell'acquisto.

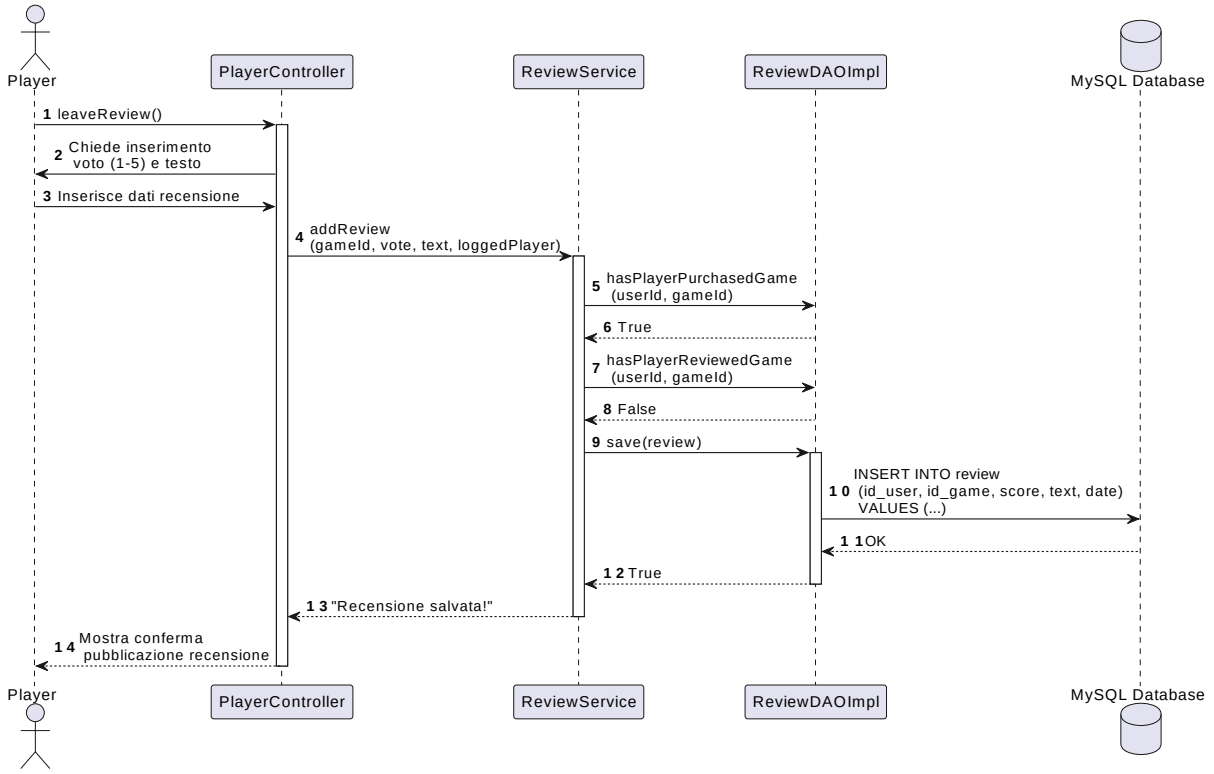


Figura 23: Diagramma di sequenza della recensione.

8 Tests

All'interno del sistema sono stati progettati, divisi su due livelli, test, che hanno permesso di collaudare e verificare la logica di business (**application**) e l'integrazione della persistenza (**persistence**):

- **Unit Test (Business Logic):** Focalizzati su servizi di livello Application (`AuthenticationServi` `CheckoutService`, `CatalogService`, `ReviewService`).
- **Integration Test (Persistenza e database reale):** Focalizzati sulle implementazioni concrete dei DAO (`UserDAOImpl`, `GameDAOImpl`, `PurchaseDAOImpl`, `ReviewDAOImpl`) interfacciati direttamente con il database.

8.1 Unit Testing

Gli Unit Test hanno lo scopo di verificare i comportamenti e le regole logiche in totale isolamento da fattori esterni come il database reale, a tal scopo è stato utilizzato un framework apposito (**Mockito**).

- **Tecnologie adottate: JUnit** per il framework di testing e **Mockito** per il mocking dei componenti di dipendenza.
- **Scenari coperti:**
  - `AuthenticationService`: Verificato il flusso di login , con gestione delle credenziali errate.

- `CheckoutService`: Verificato l'acquisto di titoli sfruttando il **Pattern Strategy** per i pagamenti (`PaymentStrategy`).
- `CatalogService`: Verificati i vincoli sui ruoli (es. solo l'amministratore può approvare i giochi) e la modifica corretta delle percentuali di sconto.
- `ReviewService`: Verificata la regola per cui un giocatore può recensire univocamente un gioco solo se precedentemente acquistato.

Come preannunciato Mockito ha permesso di effettuare un testing rapido e efficace permettendo di non agire sul database reale (quindi sporcandolo, evitando anche di dove creare un database o classi apposite), grazie all'utilizzo di *mock* (simulazioni di oggetti che si comportano allo stesso modo degli oggetti reali).

8.2 Integration Testing

Gli Integration Test hanno lo scopo di verificare la corretta esecuzione delle query SQL, la conformità dei tipi e il rispetto dei vincoli relazionali.

- **Tecnologie adottate: JUnit** , driver JDBC (per la connessione al database utilizzando la classe `DatabaseConnection`) e il database reale MySQL.
- **Gestione del Ciclo di Vita e Pulizia dei Dati:** Per garantire l'indipendenza di ogni test ed evitare la presenza di dati sporchi nel database reale, sono state utilizzate le annotazioni `@BeforeEach` e `@AfterEach` che permettono all'inizio e alla fine di ogni test di ripulire (eliminare) le voci create appositamente per ques'ultimo all'interno del database attraverso un metodo dedicato (`cleanUpTestData()`).
- **Scenari coperti:**
  - `UserDAOImpl`: Verifica del salvataggio dell'utente e della sua autenticazione.
  - `GameDAOImpl`: Verifica sul ciclo di vita del prodotto (`PENDING` → `APPROVED`), aggiornamento sconto e calcolo prezzo finale.
  - `PurchaseDAOImpl`: Verifica dell'acquisto di un gioco e relativa presenza nella sua libreria dell'utente.
  - `ReviewDAOImpl`: Verifica inserimento recensione e moderazione da parte dell'amministratore.

Test	Scenari Coperti	Framework / Tool	Esito
Unit Test	`AuthenticationService`, `CheckoutService`, `CatalogService`, `ReviewService`	JUnit , Mockito	Passato (100%)
Integration Test	`UserDAOImpl`, `GameDAOImpl`, `PurchaseDAOImpl`, `ReviewDAOImpl`	JUnit , JDBC, MySQL	Passato (100%)

Tabella 13: Tabella risultati test.

9 Codice e snippets

Qui sotto sono riportati snippets di codici vari e relativa spiegazione:

9.1 Application

- **AuthenticationService**: Qui possiamo vedere la funzione di autenticazione degli utenti attraverso verifica della password utilizzando l'algoritmo di hashing **MD5**

```
public boolean login(String email, String password) {
    User user = userDAO.findByEmail(email);
    if (user != null && PasswordHasher.hashMD5(password).equals(
        user.getPassword())) {
        this.currentUser = user;
        return true;
    }
    return false;
    }
```

- **CatalogService**: La seguente funzione ritorna tutti i videogiochi pubblicati dall'utente che l'ha chiamata (ovviamente l'utente è soggetto a verifica del ruolo, solo i **Publisher** possono avere giochi pubblicati)

```
public List<Game> getPublisherGames(User currentUser) {
        if (!(currentUser instanceof Publisher)) {
            return Collections.emptyList();
        }
        return gameDAO.findByPublisherId(currentUser.getId());
    }
```

9.2 Persistence

- **DatabaseConnection**: La funzione per recuperare l'istance della connessione al database è fatta in modo da rispettare il principio di *Singleton*, attraverso l'uso di un costruttore privato e rendendo la funzione statica e sincronizzata, in modo tale da impedire accessi contemporanei nel caso di una ipotetica aggiunta di multithreading.

```
public static synchronized DatabaseConnection getInstance() {
        try {
            if (instance == null || instance.getConnection().
                isClosed()) {
                instance = new DatabaseConnection();
            }
        } catch (SQLException e) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
```

- **GameDAOImpl**: Questa funzione ritorna l'insieme dei giochi nello stato di *pending* cioè non ancora approvati/rifiutati. Viene creata la query **SQL**, creata/recuperata

l'istanza della connessione al database ed eseguita la query e il risultato viene poi mappato negli attributi del gioco

```
public List<Game> findPending() {
        List<Game> games = new ArrayList<>();
        String query = "SELECT id, title, description, price,
           discount, state, id_publisher FROM games WHERE state =
           'PENDING'";

        try (Connection connection = DatabaseConnection.
           getInstance().getConnection();
             PreparedStatement statement = connection.
                prepareStatement(query);
             ResultSet resultSet = statement.executeQuery()) {

            while (resultSet.next()) {
                games.add(mapResultSetToGame(resultSet));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return games;
    }
```

9.3 Presentation

- **AdminController**: Questa è la funzione principale di presentazione per la dashboard dell'admin. Attraverso l'input da tastiera viene eseguita la funzione desiderata

```
public void showDashboard() {
        boolean exit = false;
        while (!exit) {
            System.out.println("\n--- DASHBOARD ADMIN ---");
            System.out.println("1. Valuta giochi in attesa");
            System.out.println("2. Modera giochi");
            System.out.println("3. Modera recensioni");
            System.out.println("0. Logout");
            System.out.print("Scelta: ");

            String choice = scanner.nextLine();
            switch (choice) {
                case "1" -> reviewPendingGames();
                case "2" -> reviewGames();
                case "3" -> reviewReviews();
                case "0" -> exit = true;
                default -> System.out.println("Scelta non valida."
                   );
            }
        }
    }
```

- **PlayerController**: La seguente funzione permette al Player di rilasciare una recensione per un gioco da esso acquistato. La funzione è resa privata perché solo

la dashboard del Player ha la possibilità di richiamarla al di fuori della classe la funzione deve rimanere irragiungibile.

```
private void leaveReview() {
        List<Game> games = catalogService.getPurchasedGames(
            loggedPlayer);
        if (games.isEmpty()) {
            System.out.println("Nessun gioco disponibile!");
            return;
        }
        games.forEach(g -> System.out.printf("[%d] %s -    %.2f\n"
            , g.getId(), g.getTitle(), g.getPrice()));
        System.out.print("\nInserisci l'ID del gioco da recensire
            (oppure 0 per tornare indietro): ");
        int gameId = Integer.parseInt(scanner.nextLine());
        if (gameId == 0)  return;
        System.out.print("Voto (1-5): ");
        int vote = Integer.parseInt(scanner.nextLine());
        System.out.print("Commento: ");
        String text = scanner.nextLine();

        System.out.println(reviewService.addReview(gameId, vote,
            text, loggedPlayer));
    }
```